

Padrões de Projeto (Design Patterns - GoF)

Parte 1: Introdução aos Padrões de Projeto

A transição da teoria dos princípios SOLID para a prática arquitetural exige um catálogo de soluções testadas e validadas pela indústria. É neste ponto que emergem os **Padrões de Projeto** (*Design Patterns*).

O conceito não se originou na computação, mas na arquitetura civil, com Christopher Alexander. Em 1994, quatro autores — Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, imortalizados como a *Gang of Four* (GoF) — publicaram o livro "*Design Patterns: Elements of Reusable Object-Oriented Software*", adaptando essa ideia para a engenharia de software.

O que são Padrões de Projeto?

Não são bibliotecas, *frameworks* ou pedaços de código prontos. São abstrações, "plantas baixas" ou modelos arquiteturais que resolvem problemas recorrentes no design de software orientado a objetos.

Por que utilizá-los?

1. **Vocabulário Ubíquo:** Permitem que desenvolvedores se comuniquem em um nível de abstração superior. Dizer "Vamos usar um *Strategy* aqui" transmite instantaneamente a estrutura, a intenção e as consequências do design, sem a necessidade de desenhar diagramas complexos.
2. **Soluções Validadas:** Evitam a "reinvenção da roda" para problemas estruturais que já foram exaustivamente resolvidos por engenheiros experientes.

Os 23 padrões originais do GoF dividem-se em três categorias funcionais:

- **Criacionais:** Lidam com mecanismos de criação de objetos de forma segura e encapsulada.
- **Estruturais:** Lidam com a composição de classes e objetos para formar estruturas maiores e complexas.
- **Comportamentais:** Lidam com a comunicação, atribuição de responsabilidades e algoritmos entre os objetos.

Parte 2: Padrões Criacionais

Em sistemas orientados a objetos primitivos, a criação de instâncias espalha-se pelo código através do operador `new`. Isso gera um acoplamento rígido (violação do DIP) entre a classe

cliente e a classe concreta instanciada. Os Padrões Criacionais abstraem esse processo de instanciação, ocultando a lógica de *como* os objetos são criados e compostos.

Abaixo, exploramos os cinco padrões criacionais clássicos e seus cenários de aplicação na engenharia moderna.

1. Singleton

Conceito: Garante que uma classe tenha apenas uma única instância em todo o ciclo de vida da aplicação e fornece um ponto global de acesso a ela. É útil quando o controle estrito sobre um recurso compartilhado é necessário.

Cenários Práticos de Utilização:

- **Cenário A: Pool de Conexões com Banco de Dados.** Abrir e fechar conexões com o banco de dados é uma operação de altíssimo custo computacional. Em vez de cada módulo do sistema instanciar sua própria conexão (esgotando os recursos do servidor de banco de dados), o sistema utiliza um `DatabaseConnectionPool` implementado como Singleton. Todos os serviços da aplicação solicitam conexões a esta única instância, que gerencia as conexões ativas de forma centralizada e segura.
- **Cenário B: Gerenciador de Configurações da Aplicação.** Sistemas corporativos leem variáveis de ambiente e arquivos de configuração (`application.properties` ou `.yaml`) na inicialização. Um `ConfigurationManager` em formato Singleton garante que o arquivo seja lido do disco apenas uma vez. Qualquer classe que precise de uma chave de configuração acessará a mesma instância em memória, evitando leituras redundantes de I/O.
- **Cenário C: Subsistema de Logging Estruturado.** Múltiplas threads e processos de um servidor web precisam gravar logs de auditoria em um mesmo arquivo físico de log de acesso. Se existissem múltiplas instâncias do gravador de log, ocorreria concorrência e corrupção do arquivo de texto. Um `AuditLogger` Singleton orquestra e enfileira essas requisições de gravação de forma thread-safe, garantindo a integridade do arquivo.

2. Factory Method (Método Fábrica)

Conceito: Define uma interface (ou classe abstrata) para criar um objeto, mas permite que as subclasses decidam qual classe concreta instanciar. Delega a responsabilidade da instanciação para as subclasses, promovendo o Princípio do Aberto/Fechado (OCP).

Cenários Práticos de Utilização:

- **Cenário A: Logística e Processamento de Fretes.** Um sistema de logística possui uma classe base `ProcessadorTransporte` com um método abstrato `criarTransporte()`. Se a regra de negócio exige transporte via caminhão, a subclasse `LogisticaTerrestre`

implementa a fábrica retornando um objeto Caminhao. Se o sistema expandir para entregas marítimas, cria-se a LogisticaMaritima que retorna um Navio. O código central que processa o envio não sabe qual veículo foi criado, apenas aciona o método entregar().

- **Cenário B: Autenticação Multi-Provedor (OAuth).** Uma aplicação web permite login social. Existe um GeradorCredenciais (Factory). Dependendo da escolha do usuário na interface, o Factory Method pode retornar um GoogleAuthenticator, um GithubAuthenticator ou um LinkedInAuthenticator. O módulo de segurança apenas chama o método comum autenticar(), sem conhecer os detalhes da API de cada rede social.
- **Cenário C: Exportação de Relatórios Dinâmicos.** Um sistema gerencial possui um botão "Exportar". A interface RelatorioFactory define o contrato. Em tempo de execução, com base na seleção do usuário, o sistema instancia um PDFRelatorioFactory ou um ExcelRelatorioFactory. O cliente recebe um produto final (Relatorio), cujo comportamento interno varia conforme a fábrica que o originou.

3. Abstract Factory (Fábrica Abstrata)

Conceito: Fornece uma interface para criar *famílias* de objetos relacionados ou dependentes sem especificar suas classes concretas. Enquanto o Factory Method cria um único produto, o Abstract Factory cria uma família inteira de produtos que precisam trabalhar juntos.

Cenários Práticos de Utilização:

- **Cenário A: Provisionamento de Infraestrutura Multicloud (IaaS).** Um sistema de orquestração de servidores precisa ser agnóstico de nuvem. Cria-se uma interface CloudFactory que exige métodos como criarServidor(), criarStorage() e criarLoadBalancer(). A implementação AWSFactory retornará instâncias de EC2, S3 e ELB. Já a AzureFactory retornará VirtualMachine, BlobStorage e AzureLB. O sistema cliente garante que não misturará um servidor da AWS com um storage da Azure, mantendo a coesão da família.
- **Cenário B: Temas de Interface Gráfica (UI Toolkits).** Uma aplicação desktop suporta os temas "Light" e "Dark". A ThemeFactory dita a criação de Botao, Janela e BarraRolagem. A DarkThemeFactory produzirá exclusivamente componentes renderizados com cores escuras, enquanto a LightThemeFactory produzirá a família de componentes claros. A aplicação renderiza a tela interagindo apenas com as interfaces genéricas dos botões e janelas.
- **Cenário C: Camada de Acesso a Dados Agrupada (Data Access Objects - DAO).** Uma arquitetura empresarial precisa suportar persistência tanto em bancos relacionais (SQL) quanto em bancos de documentos (NoSQL). A DAOFactory declara a criação de UsuarioDAO e ContratoDAO. A RelacionalDAOFactory instancia a família que utiliza JDBC/SQL. A MongoDAOFactory instancia a família que utiliza consultas BSON. O

serviço de negócios opera sobre os DAOs gerados, totalmente imune à mudança de banco.

4. Builder (Construtor)

Conceito: Separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações. É a solução ideal para resolver o "Anti-pattern do Construtor Telescópico" (classes com múltiplos construtores recebendo dezenas de parâmetros nulos).

Cenários Práticos de Utilização:

- **Cenário A: Geração de Consultas SQL Complexas (Query Builder).** Construir uma string de consulta SQL dinamicamente por concatenação gera código ilegível e propenso a falhas de segurança. Um `SqlQueryBuilder` permite montar a query em etapas lógicas: `builder.select("nome", "cpf").from("usuarios").where("idade > 18").orderBy("nome").build()`. O processo de montagem é encapsulado e apenas ao final (no `build`) o objeto final (a string SQL validada) é retornado.
- **Cenário B: Requisições HTTP em Clientes REST.** Para consumir APIs externas, uma requisição HTTP exige a configuração de múltiplos parâmetros opcionais (Headers, Tokens de Autorização, Body, Timeouts). Em vez de um construtor com 10 parâmetros, utiliza-se um `HttpRequestBuilder`. O desenvolvedor configura apenas o que precisa: `builder.comUrl(api).comMetodo("POST").comHeader("Auth", token).comCorpo(json).build()`.
- **Cenário C: Geração de Contratos Jurídicos Dinâmicos.** Em um sistema imobiliário, um contrato de locação é um objeto altamente complexo, contendo cláusulas padrão, cláusulas opcionais (fiador, seguro fiança), multas específicas e anexos. O `ContratoLocacaoBuilder` coordena a inserção de cada cláusula passo a passo. Ao final, o método `construirDocumento()` valida se todas as assinaturas e termos legais obrigatórios foram incluídos antes de instanciar o objeto Contrato final.

5. Prototype (Protótipo)

Conceito: Especifica os tipos de objetos a serem criados usando uma instância prototípica e cria novos objetos simplesmente copiando (clonando) este protótipo. É empregado quando a inicialização direta de um objeto (via banco de dados ou rede) é mais custosa do que a clonagem de um objeto existente em memória.

Cenários Práticos de Utilização:

- **Cenário A: Cache de Objetos de Alto Custo (Relatórios Analíticos).** Imagine um painel gerencial (Dashboard) que exibe as metas financeiras anuais. Recuperar e calcular esses dados no banco de dados leva 15 segundos. Ao invés de recalculá-los para cada

usuário que acessa o painel, o sistema cria o objeto DashboardEstatistico uma única vez de madrugada (o protótipo) e o armazena em memória. Durante o dia, quando um usuário solicita o relatório, o sistema simplesmente "clona" o protótipo, entregando o resultado em milissegundos.

- **Cenário B: Spawning de Entidades em Motores de Jogos.** Em um jogo em que hordas de centenas de inimigos (ex: Orcs) aparecem simultaneamente, instanciar cada um do zero — carregando malhas 3D, texturas e arquivos de áudio do disco rígido repetidas vezes — esgotaria a memória RAM e a CPU. O sistema carrega um "Orc Base" (Protótipo) na inicialização. Para gerar uma horda, o motor gráfico simplesmente clona o Orc Base e altera pequenas propriedades dos clones, como posição no mapa e barra de vida.
- **Cenário C: Templates Pré-preenchidos em Sistemas de CRM.** Um usuário precisa cadastrar dezenas de clientes corporativos (B2B) do mesmo estado, mesmo setor e mesmas condições tributárias. Em vez de preencher formulários longos em branco repetidamente, o usuário seleciona um cliente já existente e clica em "Duplicar". O padrão Prototype realiza um *deep copy* (cópia profunda) do objeto, permitindo que o usuário altere apenas o Nome e o CNPJ da nova instância.